

Transactions : Travaux pratiques

Nous proposons dans ce TP d'observer le comportement du SGBD Oracle du point de vue des propriétés transactionnelles (ACID). **Les réponses sont à saisir dans un QCM Moodle.**

Ce TP utilisera une relation de comptes bancaires que vous devrez créer (script *bank.sql*). Cette relation comprendra un numéro de compte unique (entier), le nom du propriétaire (chaîne de caractères de taille 10) et un solde (réel).

COMPTES(NC, Nom, Solde)

Sauf mention contraire, les manipulations devront être effectuées en désactivant le mode de validation automatique : vous devez entrer la commande ***set autocommit off;*** (normalement déjà par défaut) dans SQL/Plus.

Une instruction **commit** est alors nécessaire après la création de la table COMPTES et pour terminer chaque transaction.

Partie 0 : Mise en place.

1. **Connexion au serveur.** La session de travail sous le SGBD Oracle doit impérativement se faire sur le serveur im2ag-oracle. Il faut se connecter sur le serveur en ligne de commande : **ssh login@im2ag-oracle.univ-grenoble-alpes.fr** (login: le votre, mot de passe: le votre).
2. **Console Oracle.** Vous pouvez alors lancer le client Oracle en tapant la commande **gqlplus login** (login: le votre, mot de passe dans le fichier **oracle.txt** à la racine de votre **home**). Ensuite vous pouvez travailler en tapant des commandes sur la ligne de commande SQL/Plus du client Oracle.
3. **Création du schéma.** Pour créer le schéma vous pouvez vous utiliser le contenu du script **bank.sql**

Rappel: quelques commandes SQL utiles (une commande se termine par ;)

- Insertion: INSERT INTO TABLE VALUES(1, 'Paul',1000);
- Consultation: SELECT * FROM COMPTES WHERE NC=1;
- Mise à jour: UPDATE COMPTES SET SOLDE=SOLDE+10 WHERE NC=1;
- Suppression: DELETE FROM COMPTES WHERE NC=1;
- Valider une transaction: COMMIT;
- Annuler une transaction: ROLLBACK;
- Changer le mode d'isolation: SET TRANSACTION ISOLATION LEVEL SERIALIZABLE (ou READ COMMITTED)

Partie 1 : Atomicité & Durabilité.

Exercice 1 : Annulation de transaction

1. Insérez deux nouveaux comptes ayant pour propriétaire Paul (le numéro de compte et le solde sont à votre choix).
2. Affichez le contenu de la relation COMPTES.
3. Faites un *rollback*
4. Affichez le contenu de la relation COMPTES.

Exercice 2 : Validation de transaction

1. Insérez deux nouveaux comptes ayant pour propriétaire Pierre.
2. Affichez le contenu de la relation COMPTES.
3. Faites un *commit*.
4. Affichez le contenu de la relation COMPTES.
5. Faites un *rollback*
6. Affichez le contenu de la relation COMPTES. Que constatez-vous ?

Exercice 3 : Points de sauvegarde

1. Insérez un nouveau compte appartenant à Paul.
2. Placez un point de sauvegarde nommé « point1 » (*savepoint point1;*)
3. Insérez un nouveau compte appartenant à Paul.
4. Affichez le contenu de la relation COMPTES.
5. Revenez au point de sauvegarde nommé « point1 » (*rollback to point1;*)
6. Affichez le contenu de la relation COMPTES.
7. Faites un *rollback*.
8. Affichez le contenu de la relation COMPTES.

Partie 2 : Isolation.

Cette partie nécessitera d'ouvrir deux sessions (**deux terminaux ouverts**) sur le même compte de façon à permettre l'accès concurrent aux données. L'ouverture du second compte suit la même procédure qu'indiquée dans la mise en place. Ces deux sessions seront nommées S_1 et S_2 dans la suite. Attention de bien respecter la séquence des commandes sur la bonne session dans les différents exercices et de valider par COMMIT en fin de chacune des sessions, sinon le résultat ne sera pas celui escompté.

Exercice 4: Niveau d'isolation READ COMMITED (niveau par défaut)

S1	S2
1. Affichez le contenu de la relation COMPTES (SELECT)	2. Affichez le contenu de la relation COMPTES
3. Ajoutez 1000 € sur l'un des comptes de Pierre (UPDATE)	5. Affichez le contenu de la relation COMPTES
4. Affichez le contenu de la relation COMPTES	7. Affichez le contenu de la relation COMPTES
6. Validez la transaction de la session S_1	8. Validez la transaction de la session S_2

Exercice 5 : Verrouillage

Les transactions seront ici en mode READ COMMITTED (valeur par défaut d'une transaction, donc vous n'avez rien à changer). **Attention**, si rien ne se passe (blocage), NE FAITES RIEN, continuer à l'étape suivante (ordonnée de 1 à 10).

S1	S2
1. Modifier le solde d'un compte de Pierre	2. Affichez le contenu de la relation COMPTES 3. Modifiez le solde du même compte que dans S1
4. Validez la transaction de la session S ₁	5. Affichez le contenu de la relation COMPTES
6. Affichez le contenu de la relation COMPTES	7. Validez la transaction de la session S ₂
8. Affichez le contenu de la relation COMPTES	
9. Validez la transaction de la session S ₁	

Exercice 6 : Niveau d'isolation SERIALIZABLE (use case 1)

S1	S2
1. Insérez un nouveau compte appartenant à Pierre et ayant un solde de 200 € 2. Affichez le contenu de la relation COMPTES	3. passez en mode SERIALIZABLE (<i>set transaction isolation level serializable;</i>). 4. Affichez le contenu de la relation COMPTES
5. Validez la transaction de la session S ₁	6. Affichez le contenu de la relation COMPTES 7. Validez la transaction de la session S ₂ 8. Affichez le contenu de la relation COMPTES

Exercice 7 : Niveau d'isolation SERIALIZABLE (use case 2)

S1	S2
1. Ajouter un compte pour jacques 2. Validez la transaction de la session S ₁ 3. Modifier le solde du compte de jacques	4. passez en mode SERIALIZABLE (<i>set transaction isolation level serializable;</i>). 5. Affichez le contenu de la relation COMPTES 6. Modifiez aussi le compte de jacques avec une autre valeur
7. Validez la transaction de la session S ₁	8. Validez la transaction de la session S ₂ . 9. Affichez le contenu de la relation COMPTES

Exercice 8: Deadlock

S1	S2
1. Insérez un compte appartenant à Paul et ayant pour solde 300 € 2. Validez la transaction de la session S ₁ 5. Augmentez le solde du compte de Paul de 100 € 7. Affichez le contenu de la relation COMPTES 9. Diminuez le solde de Jef de 100 € 11. Validez la transaction de la session S ₁ 12. Affichez le contenu de la relation COMPTES 13. Validez la transaction de la session S ₁	3. Insérez un compte appartenant à Jef et ayant pour solde 500 €. 4. Validez la transaction de la session S ₂ . 6. Augmentez le solde de Jef de 50 €. 8. Affichez le contenu de la relation COMPTES 10. Augmentez le solde de Paul de 200 €. 14. Validez la transaction de la session S ₂ . 15. Affichez le contenu de la relation COMPTES

Partie 3 : Cohérence (Bonus)

On prend en compte une contrainte d'intégrité qui assure un solde positif ou nul (la vérification de cette contrainte aura la possibilité d'être différée, mot clé **DEFERRABLE** après la définition de la contrainte). La contrainte `soldepos` dans le fichier `bank2.sql` illustre cette contrainte.

```
create table Comptes (
    NC integer primary key,
    Nom varchar2(10),
    Solde number,
    constraint soldepos check (solde>0) deferrable
);
```

Exercice 9 : Différents scénario de gestion de contrainte

Dans la suite un seul terminal suffira. Il n'y a plus de concurrence d'accès.

- Insérez un nouveau compte appartenant à Claude et ayant un solde de 100 €.
 - Insérez un nouveau compte appartenant à Henri et ayant un solde de 200 €.
 - Validez la transaction (*commit*).
 - Passer la vérification de la contrainte sur le solde en mode immédiat (*set constraint soldepos IMMEDIATE;*).
 - Incrémentez le solde du compte de Henri de 50 €.
 - Décrémentez le solde du compte de Claude de 150 €.
 - Affichez le contenu de la relation COMPTES.
 - Annulez la transaction (*rollback*).
 - Affichez le contenu de la relation COMPTES.
- ☐ Observer l'évolution de la table `comptes` dans cette transaction.
- ☐ La contrainte est-elle violée au cours de la vie de la transaction ?
- ☐ La contrainte est-elle toujours violée à la fin de la transaction (annulation et/ou validation selon le scénario) ?

Refaire ensuite la même expérience avec les trois variantes proposées ci-après. À chaque fois, commencer par détruire les comptes de Claude et Henri (`DELETE FROM Comptes WHERE ...`) et s'assurer que l'on commence bien au sein d'une nouvelle transaction (c'est-à-dire après une commande *commit* ou *rollback*):

- contrainte en mode *IMMEDIATE* (étape 4) et validation à la fin (étape 8) ;
- contrainte en mode *DEFERRED* (étape 4) et annulation à la fin (étape 8) ;
- contrainte en mode *DEFERRED* (étape 4) et validation à la fin (étape 8).